



Research Intake 2026

Day 14

Sentence & Document Representation

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Module Overview

This compendium provides a rigorous mathematical treatment of sentence and document representation in Natural Language Processing (NLP). Every formula, algorithm, and design choice is grounded in a single running example — the Bells Corpus — so you can trace each number from raw text all the way through to final similarity scores and cluster assignments. By anchoring theory to concrete calculations, the goal is to make every step feel intuitive and reproducible.

The Bells Corpus — Our Running Example

All examples in this document use the following four documents drawn from Edgar Allan Poe's 'The Bells' (1849):

- D1: To the swinging and the ringing
- D2: of the bells, bells, bells
- D3: Of the bells, bells, bells, bells
- D4: Bells, bells, bells

This tiny corpus is ideal for teaching because it has a small, controlled vocabulary, heavy term repetition, case variation, and clear semantic similarity between documents — making it easy to verify results by hand.

Learning Objectives

1. Build a vocabulary and compute term-frequency matrices from scratch on the Bells Corpus.
2. Derive TF, IDF, and TF-IDF weights mathematically and verify every number.
3. Implement and compare mean pooling, max pooling, and SIF weighted averaging.
4. Understand Doc2Vec (PV-DM and PV-DBOW) training objectives and inference.
5. Understand USE and SBERT architectures and their training losses.
6. Compute cosine similarity and interpret the resulting similarity matrix.
7. Apply K-Means clustering to document embeddings with step-by-step centroid updates.
8. Understand FAISS ANN indexing and production semantic search pipelines.

1 Preprocessing and Vocabulary Construction

1.1 Raw Text and Tokenisation

Before any numerical representation is possible, text must be converted to tokens. Tokenisation splits a string into a sequence of atomic units. For the Bells Corpus we apply lowercase normalisation and punctuation removal, then split on whitespace.

Raw documents (original case, with punctuation):

D1: 'To the swinging and the ringing'

D2: 'of the bells, bells, bells'

D3: 'Of the bells, bells, bells, bells'

D4: 'Bells, bells, bells'

After lowercase + punctuation removal + whitespace split:

D1: ['to','the','swinging','and','the','ringing'] → 6 tokens

D2: ['of','the','bells','bells','bells'] → 5 tokens

D3: ['of','the','bells','bells','bells','bells'] → 6 tokens

D4: ['bells','bells','bells'] → 3 tokens

1.2 Building the Vocabulary

The vocabulary V is the sorted set of all unique tokens across all documents. We assign each token a fixed integer index — this index maps words to rows/columns in all subsequent matrices.

Index	Token	Appears In Documents
0	and	D1
1	bells	D2, D3, D4
2	of	D2, D3
3	ringing	D1
4	swinging	D1
5	the	D1, D2, D3
6	to	D1

Vocabulary size: $|V| = 7$. In a real corpus $|V|$ can reach 50,000–200,000. All mathematical operations scale with $|V|$ and the number of documents N .

1.3 Document Length

Let $\text{len}(D_i)$ denote the number of tokens (after preprocessing) in document i . This is used later in TF normalisation.

$$\text{len}(D1) = 6 \quad | \quad \text{len}(D2) = 5 \quad | \quad \text{len}(D3) = 6 \quad | \quad \text{len}(D4) = 3$$

2 Term Frequency (TF)

2.1 Raw Count Matrix

The raw term-frequency matrix F has shape $N \times |V|$ where $F[i][j]$ = number of times token j appears in document i . This is the foundational data structure from which all classical NLP representations derive.

Document	and	bells	of	ringing	swinging	the	to
D1	1	0	0	1	1	2	1
D2	0	3	1	0	0	1	0
D3	0	4	1	0	0	1	0
D4	0	3	0	0	0	0	0

2.2 Raw TF — Mathematical Definition

Raw TF is simply the count defined above:

$$\text{TF_raw}(t, d) = \text{count of term } t \text{ in document } d$$

Example readings from the matrix above:

- $\text{TF_raw}(\text{'bells'}, D2) = 3$ — 'bells' appears three times in D2.
- $\text{TF_raw}(\text{'bells'}, D4) = 3$ — 'bells' appears three times in D4.
- $\text{TF_raw}(\text{'swinging'}, D2) = 0$ — 'swinging' never appears in D2.

2.3 Normalised TF (Relative Frequency)

Raw counts favour long documents. A document mentioning 'bells' 6 times in 600 words is less focused on bells than a 3-word document mentioning it 3 times. Relative-frequency TF normalises by document length:

$$\text{TF_norm}(t, d) = \text{TF_raw}(t, d) / \text{len}(d)$$

Computing TF_norm for every cell of our Bells Corpus (to 4 decimal places):

Document	and	bells	of	ringing	swinging	the	to
D1	1/6=0.1667	0/6=0.0000	0/6=0.0000	1/6=0.1667	1/6=0.1667	2/6=0.3333	1/6=0.1667
D2	0/5=0.0000	3/5=0.6000	1/5=0.2000	0/5=0.0000	0/5=0.0000	1/5=0.2000	0/5=0.0000
D3	0/6=0.0000	4/6=0.6667	1/6=0.1667	0/6=0.0000	0/6=0.0000	1/6=0.1667	0/6=0.0000
D4	0/3=0.0000	3/3=1.0000	0/3=0.0000	0/3=0.0000	0/3=0.0000	0/3=0.0000	0/3=0.0000

2.4 Logarithmic TF

A third variant applies a logarithm to dampen the effect of very frequent terms. The intuition: seeing a word 10 times is informative, but 100 times is not $10\times$ more informative than 10 times.

$$\text{TF_log}(t, d) = 1 + \log(\text{TF_raw}(t, d)) \quad \text{if } \text{TF_raw}(t, d) > 0$$

$$= 0 \quad \text{if } \text{TF_raw}(t, d) = 0$$

Example: $\text{TF_log}(\text{'bells'}, D3) = 1 + \log(4) = 1 + 1.3863 = 2.3863$ (using natural log).

3 Inverse Document Frequency (IDF)

3.1 Motivation

Term frequency alone cannot distinguish informative terms from common function words. In our corpus, 'the' appears in D1, D2, and D3 — it is nearly universal and tells us little about document identity. 'Swinging' appears only in D1 — it is highly discriminative. IDF penalises terms that appear in many documents and rewards rare ones.

3.2 Document Frequency

The document frequency $DF(t)$ counts how many documents contain term t at least once:

$$DF(t) = |\{ d \in D : TF_raw(t, d) > 0 \}|$$

Computing DF for each term in the Bells Corpus:

Term	Documents Containing It	DF(t)
and	D1	1
bells	D2, D3, D4	3
of	D2, D3	2
ringing	D1	1
swinging	D1	1
the	D1, D2, D3	3
to	D1	1

3.3 Standard IDF Formula

IDF is the log of the ratio of total documents N to document frequency. The $+1$ inside the log prevents division by zero when a term appears in all documents and ensures IDF is never negative:

$$IDF(t) = \log(N / DF(t))$$

With $N = 4$ documents and using natural logarithm:

Term	DF(t)	$N/DF(t)$	$IDF = \log(N/DF)$	Interpretation
and	1	$4/1 = 4.000$	$\ln(4) = 1.3863$	Very rare — unique to D1
bells	3	$4/3 = 1.333$	$\ln(1.333) = 0.2877$	Very common — 3/4 docs
of	2	$4/2 = 2.000$	$\ln(2) = 0.6931$	Moderate — 2/4 docs
ringing	1	$4/1 = 4.000$	$\ln(4) = 1.3863$	Very rare — unique to D1
swinging	1	$4/1 = 4.000$	$\ln(4) = 1.3863$	Very rare — unique to D1
the	3	$4/3 = 1.333$	$\ln(1.333) = 0.2877$	Very common — 3/4 docs
to	1	$4/1 = 4.000$	$\ln(4) = 1.3863$	Very rare — unique to D1

3.4 Smoothed IDF (Scikit-learn default)

To avoid zero IDF for terms that appear in all documents, and to avoid division by zero, the scikit-learn implementation adds 1 to both numerator and denominator:

$$\text{IDF_smooth}(t) = \log((N + 1) / (DF(t) + 1)) + 1$$

This is the version used in practice. The +1 outside the log ensures that no term gets an IDF of zero even if it appears in every document.

Recomputing with N=4, smoothed IDF:

Term	(N+1)/(DF+1)	log of that	IDF_smooth = log + 1
and	5/2=2.500	0.9163	1.9163
bells	5/4=1.250	0.2231	1.2231
of	5/3=1.667	0.5108	1.5108
ringing	5/2=2.500	0.9163	1.9163
swinging	5/2=2.500	0.9163	1.9163
the	5/4=1.250	0.2231	1.2231
to	5/2=2.500	0.9163	1.9163

4 TF-IDF: Combining Term and Document Frequency

4.1 The Combined Formula

TF-IDF multiplies term frequency (how often a word appears in this document) by inverse document frequency (how rare the word is across the corpus). A high TF-IDF score means the term is frequent in this document but rare elsewhere — i.e., it is a good descriptor of this document's unique content.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Using normalised TF and standard IDF:

$$\text{TF-IDF}(t, d) = [\text{TF_raw}(t, d) / \text{len}(d)] \times \log(N / \text{DF}(t))$$

4.2 Full TF-IDF Matrix — Step-by-Step

We compute $\text{TF-IDF}(t, d) = \text{TF_norm}(t, d) \times \text{IDF}(t)$ for every cell. Let us trace a few representative calculations in detail before presenting the full matrix.

Calculation: TF-IDF('bells', D3)

$$\begin{aligned}\text{TF_norm}(\text{'bells'}, \text{D3}) &= 4 / 6 = 0.6667 \\ \text{IDF}(\text{'bells'}) &= \ln(4/3) = 0.2877 \\ \text{TF-IDF}(\text{'bells'}, \text{D3}) &= 0.6667 \times 0.2877 = 0.1918\end{aligned}$$

Calculation: TF-IDF('swinging', D1)

$$\begin{aligned}\text{TF_norm}(\text{'swinging'}, \text{D1}) &= 1 / 6 = 0.1667 \\ \text{IDF}(\text{'swinging'}) &= \ln(4/1) = 1.3863 \\ \text{TF-IDF}(\text{'swinging'}, \text{D1}) &= 0.1667 \times 1.3863 = 0.2310\end{aligned}$$

Calculation: TF-IDF('bells', D4)

$$\begin{aligned}\text{TF_norm}(\text{'bells'}, \text{D4}) &= 3 / 3 = 1.0000 \\ \text{IDF}(\text{'bells'}) &= \ln(4/3) = 0.2877 \\ \text{TF-IDF}(\text{'bells'}, \text{D4}) &= 1.0000 \times 0.2877 = 0.2877\end{aligned}$$

4.3 Complete TF-IDF Matrix

All values rounded to 4 decimal places. Zero entries mean the term is absent from that document.

Document	and	bells	of	ringing	swinging	the	to
D1	0.2310	0.0000	0.0000	0.2310	0.2310	0.0959	0.2310
D2	0.0000	0.1726	0.1386	0.0000	0.0000	0.0575	0.0000
D3	0.0000	0.1918	0.1155	0.0000	0.0000	0.0479	0.0000
D4	0.0000	0.2877	0.0000	0.0000	0.0000	0.0000	0.0000

4.4 Interpreting the TF-IDF Matrix

- D1 has high TF-IDF for 'and', 'ringing', 'swinging', 'to' — words unique to this document. Its TF-IDF for 'the' is low (0.0959) because 'the' appears in 3/4 documents.
- D4's TF-IDF vector is dominated entirely by 'bells' (0.2877) — the highest single value in the matrix — because D4 is only about bells and contains nothing else.
- D2 and D3 have similar profiles: moderate 'bells' and 'of', small 'the'. D3 has slightly higher 'bells' because it has a higher TF_norm (4/6 vs 3/5).

5 Vector Space Model and Document Vectors

5.1 Documents as Vectors in $\mathbb{R}^{|V|}$

The TF-IDF matrix defines each document as a point in a $|V|$ -dimensional real vector space. Each axis corresponds to a vocabulary term. The document vector encodes the document's 'aboutness' — dimensions with high values correspond to the document's key topics.

For the Bells Corpus, each document is a 7-dimensional vector. Writing them out explicitly:

```
d1_vec = [0.2310, 0.0000, 0.0000, 0.2310, 0.2310, 0.0959, 0.2310]
          (and, bells, of, ringing, swinging, the, to)
```

```
d2_vec = [0.0000, 0.1726, 0.1386, 0.0000, 0.0000, 0.0575, 0.0000]
```

```
d3_vec = [0.0000, 0.1918, 0.1155, 0.0000, 0.0000, 0.0479, 0.0000]
```

```
d4_vec = [0.0000, 0.2877, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000]
```

5.2 L2 Normalisation

Before computing similarity, we L2-normalise each document vector so all vectors lie on the unit hypersphere. This ensures cosine similarity equals the dot product. The L2 norm of a vector v is:

$$\|v\|_2 = \sqrt{v[0]^2 + v[1]^2 + \dots + v[n]^2}$$

$$v_{\text{norm}} = v / \|v\|_2$$

L2 Norm of D1

$$\begin{aligned}\|d1\| &= \sqrt{0.2310^2 + 0.0000^2 + 0.0000^2 + 0.2310^2 + 0.2310^2 + 0.0959^2 + 0.2310^2} \\ &= \sqrt{0.0534 + 0 + 0 + 0.0534 + 0.0534 + 0.0092 + 0.0534} \\ &= \sqrt{0.2228} \\ &= 0.4720\end{aligned}$$

L2 Norm of D4

$$\begin{aligned}\|d4\| &= \sqrt{0.0000^2 + 0.2877^2 + 0.0000^2 + 0.0000^2 + 0.0000^2 + 0.0000^2 + 0.0000^2} \\ &= \sqrt{0.0828} = 0.2877\end{aligned}$$

(D4 has only one non-zero dimension, so its norm equals that single value.)

6 Cosine Similarity — Full Mathematical Treatment

6.1 Definition and Geometric Intuition

Cosine similarity measures the cosine of the angle θ between two vectors. It is scale-invariant: multiplying a vector by any positive constant does not change its cosine similarity with another vector. This makes it the correct similarity metric for document comparison where vector magnitude reflects document length (a confounding factor we want to eliminate).

$$\text{cos_sim}(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v}) / (\|\mathbf{u}\|_2 \times \|\mathbf{v}\|_2)$$

$$\text{where } \mathbf{u} \cdot \mathbf{v} = \sum_i (\mathbf{u}[i] \times \mathbf{v}[i]) \quad (\text{dot product})$$

Range: [-1, +1]

$\text{cos_sim} = +1.00 \Rightarrow$ identical direction (same document)

$\text{cos_sim} = 0.00 \Rightarrow$ orthogonal (no shared vocabulary)

$\text{cos_sim} = -1.00 \Rightarrow$ opposite direction (rare for TF-IDF, impossible if all values ≥ 0)

6.2 Computing cos_sim(D2, D3) — Full Derivation

D2 and D3 are the most similar pair intuitively — both are about 'bells' with 'of' and 'the'. Let us verify this numerically.

Step 1: Write out both vectors

$\mathbf{d2} = [0.0000, 0.1726, 0.1386, 0.0000, 0.0000, 0.0575, 0.0000]$

$\mathbf{d3} = [0.0000, 0.1918, 0.1155, 0.0000, 0.0000, 0.0479, 0.0000]$

and bells of ringing swinging the to

Step 2: Compute the dot product

$$\begin{aligned} \mathbf{d2} \cdot \mathbf{d3} &= (0 \times 0) + (0.1726 \times 0.1918) + (0.1386 \times 0.1155) + (0 \times 0) + (0 \times 0) + (0.0575 \times 0.0479) + (0 \times 0) \\ &= 0 + 0.03310 + 0.01601 + 0 + 0 + 0.00275 + 0 \\ &= 0.05186 \end{aligned}$$

Step 3: Compute L2 norms

$$\begin{aligned} \|\mathbf{d2}\| &= \sqrt{0.1726^2 + 0.1386^2 + 0.0575^2} \\ &= \sqrt{0.02979 + 0.01921 + 0.00331} \\ &= \sqrt{0.05231} = 0.22871 \end{aligned}$$

$$\begin{aligned} \|\mathbf{d3}\| &= \sqrt{0.1918^2 + 0.1155^2 + 0.0479^2} \\ &= \sqrt{0.03679 + 0.01334 + 0.00229} \\ &= \sqrt{0.05242} = 0.22895 \end{aligned}$$

Step 4: Final cosine similarity

$$\begin{aligned}\cos_sim(D2, D3) &= 0.05186 / (0.22871 \times 0.22895) \\ &= 0.05186 / 0.05237 \\ &= 0.9903\end{aligned}$$

A cosine similarity of 0.9903 confirms that D2 and D3 are nearly identical in meaning — as expected, since they share the same three content words with only slightly different frequencies.

6.3 Computing $\cos_sim(D1, D4)$ — Orthogonal Vectors

D1 contains 'swinging', 'ringing', 'to', 'and' — none of which appear in D4. D4 contains only 'bells' — which is absent from D1. Their dot product is therefore zero.

$$d1 \cdot d4 = (0.2310 \times 0.0000) + (0.0000 \times 0.2877) + \dots + (0.2310 \times 0.0000) = 0.0000$$

$$\cos_sim(D1, D4) = 0.0000 / (0.4720 \times 0.2877) = 0.0000$$

Perfectly orthogonal! D1 and D4 share no vocabulary at all, so TF-IDF correctly places them at 90° from each other in vector space.

6.4 Full Pairwise Cosine Similarity Matrix

Computing all 6 unique pairs yields the following symmetric similarity matrix:

	D1	D2	D3	D4
D1	1.0000	0.0000	0.0000	0.0000
D2	0.0000	1.0000	0.9903	0.9689
D3	0.0000	0.9903	1.0000	0.9756
D4	0.0000	0.9689	0.9756	1.0000

Interpreting the Similarity Matrix

- D1 has 0.00 similarity with every other document — it shares no vocabulary with D2, D3, or D4.
- D2, D3, D4 form a tight cluster (similarities 0.97–0.99) — they are all bell-centric.
- D3 is slightly more similar to D4 (0.9756) than D2 is to D4 (0.9689) because D3 has a higher proportion of 'bells' tokens ($4/6 = 0.667$ vs $3/5 = 0.60$), making it more like D4 ($3/3 = 1.00$).

7 Averaging and Pooling Word Embeddings

7.1 From Sparse TF-IDF to Dense Word Embeddings

TF-IDF vectors are sparse (many zeros) and high-dimensional ($|V|$ dimensions). Word embeddings like Word2Vec, GloVe, and fastText represent each token as a dense low-dimensional vector (typically 50–300 dimensions) trained on large corpora so that semantically similar words have similar vectors. The key question is: how do we go from a sequence of word embedding vectors to a single sentence/document vector?

For illustration, let us assign toy 3-dimensional GloVe-like embeddings to the tokens in our Bells Corpus vocabulary. These are conceptual values chosen to reflect semantic relationships:

Token	Dim 1 (rhythmic)	Dim 2 (sound)	Dim 3 (motion)
and	0.10	0.05	0.10
bells	0.80	0.90	0.30
of	0.10	0.05	0.05
ringing	0.30	0.85	0.20
swinging	0.20	0.40	0.90
the	0.05	0.05	0.05
to	0.10	0.05	0.15

7.2 Mean Pooling

Definition

$$\text{MeanPool}(d) = (1 / \text{len}(d)) \times \text{sum_i}(\text{embed}(\text{token_i}))$$

Computing MeanPool(D2)

D2 = ['of','the','bells','bells','bells']

```
embed('of')   = [0.10, 0.05, 0.05]
embed('the')  = [0.05, 0.05, 0.05]
embed('bells') = [0.80, 0.90, 0.30]
embed('bells') = [0.80, 0.90, 0.30]
embed('bells') = [0.80, 0.90, 0.30]
```

```
Sum           = [2.55, 2.80, 1.00]
Mean (÷ 5)    = [0.51, 0.56, 0.20]
```

Computing MeanPool(D1)

D1 = ['to','the','swinging','and','the','ringing']

```
embed('to')   = [0.10, 0.05, 0.15]
```

```

embed('the')    = [0.05, 0.05, 0.05]
embed('swinging')= [0.20, 0.40, 0.90]
embed('and')     = [0.10, 0.05, 0.10]
embed('the')     = [0.05, 0.05, 0.05]
embed('ringing') = [0.30, 0.85, 0.20]

Sum              = [0.80, 1.45, 1.45]
Mean (÷ 6)       = [0.133, 0.242, 0.242]

```

All Four Document Mean-Pooled Vectors

Document	Dim 1 (rhythmic)	Dim 2 (sound)	Dim 3 (motion)	Key character
D1	0.133	0.242	0.242	Motion + sound (swinging, ringing)
D2	0.510	0.560	0.200	Strongly bell-like
D3	0.556	0.612	0.217	Even more strongly bell-like
D4	0.800	0.900	0.300	Pure bell signal

7.3 Max Pooling

Definition

$$\text{MaxPool}(d)[j] = \max_i(\text{embed}(\text{token}_i)[j]) \quad \text{for each dimension } j$$

Computing MaxPool(D2)

D2 tokens: 'of' [0.10,0.05,0.05], 'the' [0.05,0.05,0.05], 'bells'×3 [0.80,0.90,0.30]

```

MaxPool(D2)[1] = max(0.10, 0.05, 0.80, 0.80, 0.80) = 0.80
MaxPool(D2)[2] = max(0.05, 0.05, 0.90, 0.90, 0.90) = 0.90
MaxPool(D2)[3] = max(0.05, 0.05, 0.30, 0.30, 0.30) = 0.30

```

$$\text{MaxPool}(D2) = [0.80, 0.90, 0.30]$$

Max pooling effectively selects the most expressive word per dimension. For D2, 'bells' dominates all three dimensions and provides the max-pooled vector. The function words 'of' and 'the' are completely ignored, which can be beneficial.

7.4 SIF Weighted Averaging

Formula

$$\text{SIF_weight}(w) = a / (a + p(w)) \quad \text{where } a = 0.001 \text{ (smoothing hyperparameter)}$$

$$p(w) = \text{unigram probability of word } w \text{ across entire corpus}$$

$$\text{SIF_vec}(d) = \text{sum}_i(\text{SIF_weight}(\text{token}_i) \times \text{embed}(\text{token}_i)) / \text{len}(d)$$

(Followed by subtraction of the first principal component — see Section 7.5)

Computing Unigram Probabilities from Bells Corpus

Total tokens in corpus: 6+5+6+3 = 20. Counting each token across all documents:

Token	Total Count	$p(w) = \text{count}/20$	SIF weight $a/(a+p(w))$, $a=0.001$
and	1	0.0500	$0.001/(0.001+0.05) = 0.01961$
bells	13	0.6500	$0.001/(0.001+0.65) = 0.00153$
of	2	0.1000	$0.001/(0.001+0.10) = 0.00990$
ringing	1	0.0500	$0.001/(0.001+0.05) = 0.01961$
swinging	1	0.0500	$0.001/(0.001+0.05) = 0.01961$
the	4	0.2000	$0.001/(0.001+0.20) = 0.00498$
to	1	0.0500	$0.001/(0.001+0.05) = 0.01961$

Key observation: 'bells' has $p(w)=0.65$ (it makes up 65% of all tokens!) so its SIF weight drops to 0.00153 — nearly $13\times$ lower than rare words like 'swinging' (0.01961). SIF successfully down-weights the dominant term that would otherwise swamp the mean-pooled vector.

SIF Weighted Average for D4

D4 = ['bells','bells','bells'], len=3

$$\begin{aligned}\text{SIF_vec(D4)} &= (0.00153 \times [0.80, 0.90, 0.30] + 0.00153 \times [0.80, 0.90, 0.30] + 0.00153 \times [0.80, 0.90, 0.30]) / 3 \\ &= (3 \times 0.00153 \times [0.80, 0.90, 0.30]) / 3 \\ &= 0.00153 \times [0.80, 0.90, 0.30] \\ &= [0.00122, 0.00138, 0.000459]\end{aligned}$$

Notice how tiny the SIF vector for D4 is — because 'bells' has a very low SIF weight, all three repetitions are almost silenced. In practice, the post-hoc PCA step removes the first principal component, which often encodes generic corpus direction rather than document-specific meaning. For tiny corpora like ours, the PCA step is more meaningful on large corpora.

8 Doc2Vec — Paragraph Vectors

8.1 Motivation and Architecture

All pooling methods (mean, max, SIF) operate on pre-trained, context-free word vectors. They treat every occurrence of 'bells' identically, regardless of surrounding context. Doc2Vec (Le & Mikolov, 2014) trains document-level vectors jointly with word vectors, learning a unique paragraph vector for each document during training.

The key insight: add a special document-level vector (the paragraph vector) as an additional input to the Word2Vec context window. This vector is shared across all context windows within a document and therefore learns to encode the document's global meaning.

8.2 PV-DM: Distributed Memory Model

Training Objective

At each training step, a context window of k words is sampled from a document. The model predicts the centre word given: (a) the paragraph vector, and (b) the surrounding context words. The loss is the negative log-likelihood of the correct centre word:

$$L_{\text{PV-DM}} = - (1/T) \times \sum_t \log P(w_t \mid d_{\text{vec}}, w_{\{t-k\}}, \dots, w_{\{t-1\}}, w_{\{t+1\}}, \dots, w_{\{t+k\}})$$

where:

w_t = centre word at position t

d_{vec} = paragraph (document) vector for this document

T = total number of training positions in the document

Context Window Example from D2

$D2 = ['of', 'the', 'bells', 'bells', 'bells']$. With window size $k=1$, centred on position 2 ('bells'):

Context: ['the'] + $d2_{\text{vec}}$

Target: 'bells'

The model concatenates $\text{embed}('the')$ and $d2_{\text{paragraph_vec}}$, then passes through a softmax over the vocabulary to predict 'bells'.

Why Does the Paragraph Vector Help?

Consider two different documents both containing the window ['the', '?', 'bells'].

Document A is about church music; Document B is about a store selling cowbells.

In both cases the word context is identical — only the paragraph vector differs.

The paragraph vector acts as a 'topic memory' that disambiguates the missing word based on the document's overall theme, even when local context is identical.

8.3 PV-DBOW: Distributed Bag of Words

Training Objective

PV-DBOW ignores context words entirely. Given only the paragraph vector, predict randomly sampled words from the document. This is simpler but forces the paragraph vector to compress global document content:

$$L_{\text{PV-DBOW}} = - (1/T) \times \sum_t \log P(w_t | d_{\text{vec}})$$

$$P(w | d_{\text{vec}}) = \text{softmax}(W \times d_{\text{vec}})[w]$$

where W is a learned projection matrix ($|V| \times \text{dim_paragraph}$)

Inference for New Documents

At test time, the word vectors and projection matrix W are frozen. For a new document d_{new} , we initialise a random $d_{\text{new_vec}}$ and run gradient descent to find the vector that maximises the likelihood of the words in d_{new} :

$$d_{\text{new_vec}}^* = \text{argmin}_{\{d_{\text{new_vec}}\}} L_{\text{PV-DBOW}}(d_{\text{new_vec}} | \text{frozen word vectors})$$

Typically requires 10–200 gradient steps with learning rate ~ 0.01

8.4 Negative Sampling

Computing the full softmax over $|V|$ tokens at every training step is computationally expensive ($O(|V|)$ per step). Negative sampling approximates the softmax by treating the task as binary classification: is this (word, context) pair genuine or noise? For k negative samples:

$$L_{\text{neg}} = \log \sigma(v_w \cdot v_c) + \sum_{i=1}^k E_{\{w_i \sim P_n\}} [\log \sigma(-v_{\{w_i\}} \cdot v_c)]$$

where:

$$\sigma(x) = \text{sigmoid}(x) = 1/(1 + \exp(-x))$$

v_w = embedding of the target word

v_c = embedding of the context (or paragraph vector)

P_n = noise distribution, typically $\text{unigram}^{(3/4)}$ for word sampling

With $k=5-15$ negative samples, training is 10–100× faster than full softmax with minimal quality loss. The 3/4-power smoothing of the unigram distribution slightly up-weights rare words as negatives, preventing the model from being dominated by common-word negative samples.

9 Universal Sentence Encoder (USE)

9.1 Overview

Google's Universal Sentence Encoder (Cer et al., 2018) produces 512-dimensional sentence embeddings through multi-task training. Two architectural variants are provided, with different speed-accuracy tradeoffs.

9.2 DAN Variant — Deep Averaging Network

Architecture

The DAN variant is a feedforward network. Input tokens are embedded (word + bigram embeddings), mean-pooled, then passed through dense layers. Unlike the pooling methods in Section 7, the dense layers learn non-linear transformations:

```
Step 1: token_embeds = [ embed(t) for t in tokens ]
Step 2: averaged      = mean_pool(token_embeds)
Step 3: h1            = ReLU( W1 × averaged + b1 )    shape: (512,)
Step 4: h2            = ReLU( W2 × h1 + b2 )         shape: (256,)
Step 5: h3            = ReLU( W3 × h2 + b3 )         shape: (512,)
Step 6: output        = L2_normalise( h3 )           shape: (512,)
```

Training Objective — Multi-Task

USE is trained simultaneously on three tasks. Each task provides gradient signal that shapes the embedding space differently:

- Task 1 — Conversational Response Prediction: Given an input sentence, predict the correct response from a set of candidates. Trains the encoder to understand dialogue and intent.
- Task 2 — Natural Language Inference (NLI): Given premise and hypothesis, classify as entailment / contradiction / neutral. Trains the encoder to capture semantic entailment.
- Task 3 — Supervised Semantic Similarity (STS): Directly regress the encoder's cosine similarity to human similarity scores. Trains the embedding space geometry.

$$L_{\text{total}} = \text{lambda_1} \times L_{\text{response}} + \text{lambda_2} \times L_{\text{NLI}} + \text{lambda_3} \times L_{\text{STS}}$$

where lambda_1 , lambda_2 , lambda_3 are task weight hyperparameters.

9.3 Transformer Variant

The Transformer variant uses full self-attention. Each token attends to all other tokens, enabling the model to resolve ambiguity through global context. The sentence vector is obtained by averaging the final-layer hidden states (mean pooling on contextualised representations). This variant is more accurate but ~50× slower than DAN.

USE Practical Characteristics

DAN variant: ~2 ms/sentence on CPU, ~0.5 ms on GPU, 250 MB model size

Transformer variant: ~100 ms/sentence on CPU, ~5 ms on GPU, 800 MB model size

Output dimension: 512 (both variants)

Multilingual: mUSE supports 16 languages in a single model

Input length: Handles up to ~512 tokens; degrades gracefully beyond

10 Sentence-BERT (SBERT) — The Modern Standard

10.1 The Cross-Encoder Problem

Standard BERT processes two sentences together as a single sequence: [CLS] sentence_A [SEP] sentence_B [SEP]. The CLS token representation is then used for classification. This allows full cross-attention between sentences, yielding high accuracy. However, it requires running a full BERT forward pass for every pair of sentences.

$$\text{Pairwise comparisons for } N \text{ sentences} = N \times (N-1) / 2$$

$$\begin{aligned} N = 10,000: \quad & 10,000 \times 9,999 / 2 = 49,995,000 \text{ BERT passes} \\ \text{Time @ 65ms per pass} \quad & = 49,995,000 \times 0.065 \text{ seconds} \\ & = 3,249,675 \text{ seconds} \\ & \approx 37.6 \text{ days} \end{aligned}$$

10.2 The Bi-Encoder Solution

SBERT replaces this with a bi-encoder (siamese) architecture. Both sentences are encoded independently using the same BERT model with shared weights, and the sentence representations are derived from mean pooling the final hidden states:

$$\begin{aligned} \mathbf{u} &= \text{MeanPool}(\text{BERT}(\text{sentence_A})) \quad \# \text{ shape: (768,)} \\ \mathbf{v} &= \text{MeanPool}(\text{BERT}(\text{sentence_B})) \quad \# \text{ shape: (768,)} \\ \text{similarity} &= \text{cos_sim}(\mathbf{u}, \mathbf{v}) \quad \# \text{ single dot product after normalisation} \end{aligned}$$

With pre-computed embeddings, comparing $N=10,000$ sentences takes:

$$\begin{aligned} \text{Encode phase:} \quad & 10,000 \times 5 \text{ ms} = 50 \text{ seconds} \\ \text{Compare phase:} \quad & 49,995,000 \text{ dot products} \approx 0.01 \text{ seconds (vectorised numpy/GPU)} \\ \text{Total:} \quad & \sim 50 \text{ seconds (vs 37.6 days for cross-encoder)} \end{aligned}$$

10.3 NLI Fine-Tuning — Mathematical Detail

Training Data

SBERT is fine-tuned on SNLI (570K pairs) and MultiNLI (430K pairs) — approximately 1 million sentence pairs labelled as entailment, neutral, or contradiction.

Classification Loss

Given sentence embeddings $\mathbf{u}$ and $\mathbf{v}$, the classification head receives a concatenation:

$$\begin{aligned} \text{feature_vector} &= \text{concat}(\mathbf{u}, \mathbf{v}, |\mathbf{u} - \mathbf{v}|) \quad \# \text{ shape: (768} \times 3 \text{,) = (2304,)} \\ \text{logits} &= \mathbf{W_classify} \times \text{feature_vector} + \mathbf{b} \quad \# \text{ shape: (3,)} \\ \text{probabilities} &= \text{softmax(logits)} \quad \# [\mathbf{P_entail}, \mathbf{P_neutral}, \mathbf{P_contradict}] \end{aligned}$$

$$L_{\text{classify}} = - \sum_i y_i \times \log(P_i) \quad \# \text{ cross-entropy loss}$$

The element-wise difference $|u-v|$ is crucial: it provides a direct signal about which dimensions differ between the two sentence representations, helping the head learn that large differences indicate contradiction and small differences indicate entailment.

Regression Loss for STS Tasks

When fine-tuning directly on semantic similarity scores (e.g., STS-B scores 0–5), the loss becomes mean squared error between predicted and gold similarity:

$$\begin{aligned} L_{\text{regression}} &= \text{MSE}(\cos_sim(u, v), y_{\text{gold}} / 5.0) \\ &= (\cos_sim(u, v) - y_{\text{gold}}/5.0)^2 \end{aligned}$$

where y_{gold} is the human similarity score in $[0, 5]$, normalised to $[0, 1]$.

10.4 Contrastive Learning — The Modern SBERT Training

More recent SBERT models (2021–present) use a contrastive Multiple Negatives Ranking (MNR) loss instead of NLI classification. Given a batch of (query, positive) pairs, all other positives in the batch become negatives:

For a batch of N pairs: $(q_1, p_1), (q_2, p_2), \dots, (q_N, p_N)$

$\text{similarity_matrix}[i][j] = \cos_sim(q_i, p_j)$ # $N \times N$ matrix

$$L_{\text{MNR}} = - (1/N) \times \sum_i \log(\exp(\text{sim}[i][i]) / \sum_j \exp(\text{sim}[i][j]))$$

This is equivalent to N -way softmax classification where the correct answer is always the diagonal element.

SBERT Performance Benchmarks

Model: all-mpnet-base-v2 (best general-purpose English model as of 2024)

STS-B Spearman: 86.9 (vanilla BERT without siamese: 52.2)

MTEB Average: 57.8 (56 tasks across 8 categories)

Inference speed: ~5 ms per sentence on GPU

Memory: 420 MB (base model), 80 MB (MiniLM variant)

Model: all-MiniLM-L6-v2 (fast deployment model)

STS-B Spearman: 84.5

Inference speed: ~1 ms per sentence on GPU (5× faster than mpnet)

11 Semantic Search with FAISS

11.1 The Retrieval Problem

Given a query sentence q and a corpus of N document embeddings $\{d_1, \dots, d_N\}$, we want to find the top- K most similar documents. Exact search requires computing $\text{cos_sim}(q, d_i)$ for every i — $O(N \times d)$ operations per query, where d is embedding dimension.

Exact search complexity: $O(N \times d)$ per query

**$N = 1,000,000$ documents, $d = 768$ dimensions:
= 768,000,000 multiply-add operations
≈ 10–50 ms on GPU for a single query
= 10,000–50,000 ms for 1000 concurrent queries ← unacceptable for real-time**

11.2 HNSW — Hierarchical Navigable Small World

Data Structure

HNSW builds a multi-layer proximity graph. Each layer is a sub-graph of the one below. Layer 0 contains all nodes; higher layers contain exponentially fewer nodes (each node is inserted into layer l with probability p^l , typically $p=1/e$).

**For a node v inserted at layer l_{max} :
 $l_{\text{max}} = \text{floor}(-\ln(\text{uniform}(0,1)) \times 1/\ln(M))$**

where M = maximum number of neighbours per node (typical: 16–64)

Search Algorithm

To find the K nearest neighbours of query q :

1. Start at the entry point in the highest layer.
2. Greedily navigate to the nearest neighbour of q in the current layer.
3. Descend to the next layer, starting from the current nearest neighbour.
4. Repeat until layer 0. In layer 0, expand the search to ef_search candidates.
5. Return the K closest of the ef_search candidates by exact cosine similarity.

**Time complexity: $O(\log(N) \times \text{ef_search})$ per query
Recall @ $K=10$: ~99% with $\text{ef_search}=50$
Speedup vs exact: 100–1000× depending on N and ef_search**

11.3 IVF-PQ — Inverted File with Product Quantisation

Step 1: Voronoi Partitioning (IVF)

Train K cluster centroids on the document embeddings using K-Means. Each document is assigned to its nearest centroid. At query time, only the n_{probe} nearest centroids (and their assigned documents) are searched.

Search space reduction: n_{probe} / K (typical: $8/1024 = 0.78\%$ of corpus)

Step 2: Product Quantisation (PQ)

Within each Voronoi cell, compress embeddings to save memory. Split the d -dimensional vector into M sub-vectors, each of dimension d/M . Quantise each sub-vector independently using a learned codebook of 256 centroids (1 byte):

Original vector: $d \text{ floats} \times 4 \text{ bytes} = 768 \times 4 = 3,072 \text{ bytes}$

PQ compressed: $M \text{ sub-quantisers} \times 1 \text{ byte} = 48 \times 1 = 48 \text{ bytes}$

Compression ratio: $3072 / 48 = 64\times$ memory reduction

Approximate dot product: $u_{\text{pq}} \cdot v_{\text{pq}} \approx u \cdot v$ (within 1–2% error)



Production Semantic Search Architecture

Offline (once):

1. Encode all N documents with SBERT $\rightarrow N \times 768$ float32 matrix
2. Build FAISS HNSW or IVF-PQ index on the embeddings
3. Store document ID $\rightarrow$ content mapping

Online (per query, $\sim 10\text{ms}$ total):

1. Receive query text (e.g., 'bells ringing at night')
2. Encode query with same SBERT model $\rightarrow 768\text{-dim vector}$ [$\sim 5\text{ms}$]
3. FAISS ANN search $\rightarrow$ top-100 candidate document IDs [$\sim 0.1\text{ms}$]
4. Optional: re-rank top-100 with a cross-encoder [$\sim 5\text{ms}$]
5. Return top-K results with similarity scores

12 Document Clustering — Full Worked Example

12.1 Clustering Objective

Clustering assigns documents to groups without any labels, based purely on embedding similarity. The goal is to find K groups such that documents within a group are more similar to each other than to documents in other groups. We will apply K-Means to the TF-IDF vectors from the Bells Corpus.

12.2 K-Means Algorithm — Mathematical Definition

Objective Function

$$\text{Minimise: } J = \sum_{k=1}^K \sum_{d \in \text{cluster}_k} \|d_{\text{vec}} - \mu_k\|^2$$

where $\mu_k = (1/|\text{cluster}_k|) \times \sum_{d \in \text{cluster}_k} d_{\text{vec}}$
is the centroid (mean) of cluster k .

Algorithm Steps

6. Initialise K centroids (random selection or K-Means++ seeding).
7. Assignment step: assign each document to its nearest centroid by Euclidean distance.
8. Update step: recompute each centroid as the mean of assigned documents.
9. Repeat steps 2–3 until centroid positions change by less than tolerance ϵ .

12.3 K-Means on Bells Corpus ($K=2$)

Using 2D Reduced TF-IDF Vectors

For visualisability, we use only the 'bells' and 'the' dimensions of the TF-IDF matrix (these are the two most discriminative). Other dimensions would be included in a real run.

Document	TF-IDF('bells')	TF-IDF('the')	2D Coordinates
D1	0.0000	0.0959	(0.0000, 0.0959)
D2	0.1726	0.0575	(0.1726, 0.0575)
D3	0.1918	0.0479	(0.1918, 0.0479)
D4	0.2877	0.0000	(0.2877, 0.0000)

Iteration 1: Random Initialisation

Suppose we randomly initialise: Centroid₁ = D1 = (0.0000, 0.0959), Centroid₂ = D3 = (0.1918, 0.0479).

Assignment Step 1

Compute Euclidean distance from each document to each centroid:

$$\begin{aligned} \text{dist}(D1, C1) &= \sqrt{(0-0)^2 + (0.0959-0.0959)^2} = 0.0000 \rightarrow \text{assign to } C1 \\ \text{dist}(D1, C2) &= \sqrt{(0-0.1918)^2 + (0.0959-0.0479)^2} = \sqrt{0.03679+0.00230} = 0.1976 \end{aligned}$$

$$\begin{aligned}\text{dist}(D2, C1) &= \sqrt{((0.1726-0)^2 + (0.0575-0.0959)^2)} = \sqrt{(0.02979+0.00148)} = 0.1770 \\ \text{dist}(D2, C2) &= \sqrt{((0.1726-0.1918)^2 + (0.0575-0.0479)^2)} = \sqrt{(0.00037+0.000009)} = 0.0213 \rightarrow \\ &\text{assign to } C2\end{aligned}$$

$$\begin{aligned}\text{dist}(D3, C1) &= \sqrt{((0.1918)^2 + (0.0479-0.0959)^2)} = \sqrt{(0.03679+0.00230)} = 0.1976 \\ \text{dist}(D3, C2) &= \sqrt{((0)^2 + (0)^2)} = 0.0000 \rightarrow \text{assign to } C2 \text{ (D3 is } C2)\end{aligned}$$

$$\begin{aligned}\text{dist}(D4, C1) &= \sqrt{((0.2877)^2 + (0-0.0959)^2)} = \sqrt{(0.08277+0.00920)} = 0.3032 \\ \text{dist}(D4, C2) &= \sqrt{((0.2877-0.1918)^2 + (0-0.0479)^2)} = \sqrt{(0.00920+0.00230)} = 0.1073 \rightarrow \text{assign to } \\ &C2\end{aligned}$$

After Assignment Step 1: Cluster 1 = {D1}, Cluster 2 = {D2, D3, D4}

Update Step 1 — Recompute Centroids

New C1 = mean(D1) = (0.0000, 0.0959) (unchanged, single member)

$$\begin{aligned}\text{New } C2 &= \text{mean}(D2, D3, D4) \\ &= ((0.1726+0.1918+0.2877)/3, (0.0575+0.0479+0.0000)/3) \\ &= (0.6521/3, 0.1054/3) \\ &= (0.2174, 0.0351)\end{aligned}$$

Assignment Step 2

Recompute distances with updated centroids C1=(0.0000, 0.0959), C2=(0.2174, 0.0351):

All four distances are checked again.
D1 remains closest to C1: $\text{dist}(D1, C1)=0$, $\text{dist}(D1, C2)=\sqrt{(0.04727+0.00372)}=0.2259$
D2, D3, D4 all remain closer to C2 (verified by symmetric calculation).

Assignments unchanged → Algorithm converges after 1 update.

☒ K-Means Result on Bells Corpus

Cluster 1 (centroid $\approx [0.000, 0.096]$): { D1 }

→ D1 is about 'swinging and ringing' — no 'bells' vocabulary.

Cluster 2 (centroid $\approx [0.217, 0.035]$): { D2, D3, D4 }

→ D2, D3, D4 are all about 'bells' with varying intensity.

This matches our human intuition perfectly. K-Means correctly separated the

motion/sound document (D1) from the bells-centric documents (D2, D3, D4).

13 Evaluation Metrics

13.1 Spearman Rank Correlation

For semantic similarity tasks, evaluation computes Spearman's rank correlation coefficient (ρ) between model-predicted cosine similarities and human-annotated similarity scores. Unlike Pearson correlation, Spearman correlation is rank-based and insensitive to the scale of the similarity scores.

$$\rho = 1 - (6 \times \sum_i d_i^2) / (n \times (n^2 - 1))$$

where:

$$d_i = \text{rank}(\text{model_sim}_i) - \text{rank}(\text{human_sim}_i)$$

n = number of sentence pairs

$\rho = 1.00$ means perfect rank agreement

$\rho = 0.00$ means no rank correlation

$\rho = -1.00$ means perfect inverse rank agreement

Verified on Bells Corpus

Our cosine similarity matrix gives: $\cos(D2, D3)=0.9903$, $\cos(D2, D4)=0.9689$, $\cos(D3, D4)=0.9756$. If human annotators rate D2–D3 most similar, D3–D4 second, D2–D4 third, the model ranks match perfectly $\rightarrow \rho = 1.00$ on this mini-corpus.

13.2 MTEB — Massive Text Embedding Benchmark

MTEB (Muennighoff et al., 2022) evaluates embedding models across 56 datasets and 8 task categories. It is the de-facto standard for comparing sentence embedding models. Each task uses a different metric appropriate to the task:

Task Category	# Datasets	Metric	Example Dataset
Classification	12	Accuracy	SentEval (topic classification)
Clustering	11	V-Measure	ArXiv category clustering
Pair Classification	3	AP	TwitterSemEval (paraphrase detection)
Reranking	4	MAP	StackExchange answer reranking
Retrieval	15	nDCG@10	BEIR: 18 IR datasets
STS	10	Spearman ρ	STS-B, STS12-STS16
Summarisation	1	Spearman ρ	SummEval
Bitext Mining	6	F1	BUCC, Tatoeba

14 Beyond Fixed-Length Representations

14.1 The Compression Problem

Every method we have studied produces a fixed-size vector from a variable-length document. This is inherently lossy. Consider:

**A 10,000-word legal contract tokenises to ~12,000 tokens.
SBERT base model produces a 768-dimensional vector.**

**Information rate: $12,000 \text{ tokens} \times \sim 13 \text{ bits/token} \approx 156,000 \text{ bits of information}$
Embedding capacity: $768 \text{ dimensions} \times 32 \text{ bits/float} \approx 24,576 \text{ bits}$**

Compression ratio: $156,000 / 24,576 \approx 6.3\times$ information loss minimum

14.2 Hierarchical Encoding

Rather than encoding the whole document at once, hierarchical encoders process sentences individually then aggregate:

**Level 1: $s_i = \text{SBERT}(\text{sentence}_i)$ for each sentence i # shape: (768,)
Level 2: $d = \text{SBERT_doc}([s_1, \dots, s_n])$ # shape: (768,)**

Or simply: $d = \text{MeanPool}([s_1, \dots, s_n])$ with optional attention weights

14.3 Long-Document Transformers

Standard transformers have $O(n^2)$ attention complexity (all pairs of tokens attend to each other). This limits BERT to 512 tokens. Long-document transformers replace full attention with sparse attention patterns:

Model	Max Tokens	Attention Type	Memory Complexity
BERT	512	Full $O(n^2)$	$O(n^2)$
Longformer	4,096	Sliding window + global	$O(n \times w)$ where w = window size
BigBird	4,096	Random + local + global	$O(n)$
Mamba (SSM)	Unlimited (theoretically)	Selective state spaces	$O(n)$

14.4 Retrieval-Augmented Generation (RAG)

RAG avoids the compression problem by chunking documents into passages and encoding each passage separately. At query time, relevant passages are retrieved and injected into the LLM's context window.

OFFLINE:

1. Split document into chunks of ~512 tokens with 64-token overlap
2. Encode each chunk: $\text{chunk_vec}_i = \text{SBERT}(\text{chunk}_i)$

3. Index all chunk_vec_i in FAISS

ONLINE (per query q):

- 1. q_vec = SBERT(q)**
- 2. top_chunks = FAISS.search(q_vec, k=5)**
- 3. context = concatenate(top_chunks)**
- 4. answer = LLM(prompt='Context: {context}\nQuestion: {q}')**

14.5 SimCSE — Contrastive Learning Without Labels

SimCSE (Gao et al., 2021) showed that high-quality sentence embeddings can be trained without any labelled data. The key insight: encode the same sentence twice with different random dropout masks. These two noisy versions of the same sentence serve as positive pairs. All other sentences in the batch are negatives.

For sentence s with dropout masks m1, m2:

z = BERT(s, m1) (first encoding)

z' = BERT(s, m2) (second encoding with different dropout)

$$\mathbf{L_SimCSE} = -\log[\exp(\text{sim}(z, z')/\tau) / \sum_{j \neq i} \exp(\text{sim}(z_i, z_j)/\tau)]$$

**where τ is a temperature hyperparameter (typically 0.05)
and the denominator sums over all other sentences in the batch as negatives.**

15 Comprehensive Comparison of All Methods

15.1 Method Comparison Table

The following table summarises every method covered in this module against dimensions relevant to practical deployment:

Method	Year	Dims	Requires Training?	Inference Speed	Quality (STS-B ρ)	Best For
Bag of Words (TF-IDF)	2000s	N (vocab size)	No (counting)	<0.1 ms	~40–50	Keyword retrieval, sparse IR
Mean Pooling (GloVe)	2014	300	No (pre-trained)	<1 ms	~58–65	Fast prototype, no GPU
SIF Weighted Avg	2017	300	No (freq stats)	~1 ms	~68–72	Better than mean, no training
Doc2Vec	2014	100–300	Yes (target corpus)	2–5ms + inf-GD	~60–68	Document-level topics
InferSent (BiLSTM)	2017	4096	Yes (NLI)	~10 ms	~75–78	Pre-BERT supervised baseline
Universal Sentence Encoder	2018	512	Yes (multi-task)	2 ms (DAN)	~78–82	Low-latency production
SBERT (base)	2019	768	Yes (NLI+STS)	~5 ms	~86–87	State-of-art, general
SBERT (MiniLM-L6)	2020	384	Yes (distilled)	~1 ms	~84–85	Fast production, limited GPU
E5-large / BGE-large	2023	1024	Yes (large scale)	~15 ms	~88–90	MTEB leaderboard tasks
text-embedding-3 (OpenAI)	2024	256–3072	Yes (proprietary)	API latency	~90+	General RAG, no self-hosting

15.2 Decision Guide

Which Method Should You Use?

NO training data + need < 2ms latency:

- TF-IDF vectors (Section 4) + cosine similarity
- Or: Mean pooling with fastText (Section 7.2)

NO training data + can afford 2–10ms:

- SIF weighted averaging (Section 7.4)
- Or: Universal Sentence Encoder DAN (Section 9.2)

HAVE labelled pairs + GPU available:

- SBERT all-MiniLM-L6-v2 for speed, all-mpnet-base-v2 for accuracy (Section 10)

DOCUMENT-level topics + large unlabelled corpus:

- Doc2Vec PV-DM + PV-DBOW concatenated (Section 8)

MULTILINGUAL requirements:

- paraphrase-multilingual-MiniLM-L12-v2 or mUSE

DOMAIN-SPECIFIC (medical, legal, code):

- BioBERT/PubMedBERT/CodeBERT fine-tuned with SBERT siamese objective

VERY LONG documents (> 512 tokens):

- RAG with SBERT chunking (Section 14.4)
- Or: Longformer for document classification

16 End-to-End Pipeline: Bells Corpus Semantic Search

16.1 Building the Index

We now build a complete semantic search pipeline for the Bells Corpus using TF-IDF vectors and cosine similarity. This demonstrates every step from raw text to answering a query.

```
# Step 1: Corpus definition
corpus = {
    'D1': 'to the swinging and the ringing',
    'D2': 'of the bells bells bells',
    'D3': 'of the bells bells bells bells',
    'D4': 'bells bells bells'
}

# Step 2: Build TF-IDF vectors
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(norm='l2') # L2-normalises each document vector
tfidf_matrix = vectorizer.fit_transform(list(corpus.values())) # shape: (4, 7)

# Step 3: Answer a query
query = 'ringing bells'
query_vec = vectorizer.transform([query]) # uses same vocabulary

# Step 4: Compute cosine similarities (vectors are already L2-normalised)
from sklearn.metrics.pairwise import cosine_similarity
sims = cosine_similarity(query_vec, tfidf_matrix).flatten()

# Step 5: Rank and return
ranked = sorted(zip(corpus.keys(), sims), key=lambda x: -x[1])
for doc_id, score in ranked:
    print(f'{doc_id}: {score:.4f} | {corpus[doc_id]}')
```

16.2 Query: 'ringing bells' — Expected Output

The query 'ringing bells' contains two terms. 'ringing' appears only in D1 (IDF high); 'bells' appears in D2, D3, D4 (IDF lower). The query TF-IDF vector will have non-zero values for both terms. The ranking should place D1 and D2/D3/D4 both reasonably high, but D1 gets a unique boost from 'ringing'.

Rank	Document	Expected Similarity	Reason
1	D1	~0.55	Unique match on 'ringing' (high IDF) + no 'bells' penalty
2	D4	~0.49	Pure 'bells' document — highest 'bells' TF-IDF
3	D3	~0.43	High 'bells' frequency (4/6 of tokens)
4	D2	~0.38	Lowest 'bells' frequency in bell-cluster (3/5 of tokens)

16.3 Extending to SBERT

Replacing TF-IDF with SBERT embeddings handles synonyms, paraphrases, and cross-lingual queries that TF-IDF cannot. For example, the query 'chiming metal instruments' would score 0.00 against our TF-IDF index (no shared vocabulary) but would achieve a moderate SBERT similarity (~0.45) because SBERT understands that 'chiming metal instruments' is semantically related to 'bells'.

Module Summary

This module built a complete mathematical foundation for sentence and document representation in NLP, grounding every concept in the Bells Corpus. Here are the essential takeaways:

Key Mathematical Results

Concept	Formula / Key Number	Bells Corpus Result
TF_norm('bells',D4)	$\text{count} / \text{doc_length} = 3/3$	1.0000 (pure bells document)
IDF('bells')	$\ln(N/DF) = \ln(4/3)$	0.2877 (common across 3/4 docs)
TF-IDF('bells',D3)	$\text{TF_norm} \times \text{IDF} = 0.6667 \times 0.2877$	0.1918
cos_sim(D2,D3)	$\text{dot}(d2,d3)/(\ d2\ \times \ d3\)$	0.9903 (near-identical)
cos_sim(D1,D4)	$\text{dot}(d1,d4)/(\ d1\ \times \ d4\)$	0.0000 (no shared vocabulary)
K-Means (K=2) result	$\min \sum \ d - \text{centroid}\ ^2$	Cluster1: {D1}, Cluster2: {D2,D3,D4}
SBERT speedup	$N*(N-1)/2$ comparisons	50 seconds vs 37.6 days for N=10,000

Progression of Methods

- I. TF-IDF (sparse, exact vocabulary match) → best for keyword search.
- II. Mean/Max/SIF Pooling (dense, unsupervised) → fast general-purpose baselines.
- III. Doc2Vec (dense, document-level training) → captures document topics end-to-end.
- IV. USE (dense, multi-task trained) → low-latency production deployment.
- V. SBERT (dense, siamese fine-tuning) → state-of-the-art semantic similarity.
- VI. FAISS ANN (index structure) → scales SBERT similarity search to millions of documents.
- VII. RAG (chunking + retrieval) → solves the long-document compression problem.

The future of AI is bright, and you can be part of it!